VESTRA

v4.0.0 — Complete System Guide

AI-Powered Real Estate Ecosystem for Africa

Property	Value
Document Version	1.0
System Version	4.0.0
Generated	June 21, 2026
Environment	Production-ready
Support	support@vestra.co.ke

Table of Contents

1. System Overview

VESTRA is an AI-powered real estate ecosystem designed specifically for Kenya and Africa. It provides a complete platform for property discovery, AI-powered verification, rent payment, property buying/selling, agent connections, and landlord portfolio management. The system combines blockchain-like title verification, M-Pesa mobile money integration, and cutting-edge AI to create the most trusted property platform in Africa.

Tech Stack

Component	Technology	Version
Backend Framework	FastAPI (Python)	0.115+
Frontend Framework	Next.js (React)	16.2.9
Database	PostgreSQL	16 Alpine
Cache / Sessions	Redis	7 Alpine
Reverse Proxy	Nginx	1.27 Alpine
Monitoring	Prometheus + Grafana	3.1 / 11.4
Alerting	Alertmanager	0.28
Container Runtime	Docker + Docker Compose	Latest
AI Engine	Vestra AI (rule-based)	Custom
Payments	M-Pesa, Stripe, PayPal, Bank, Crypto	Multi

Docker Services (12 services)

Service	Role	Port
postgres	Primary database	5432 (internal)
redis	Caching, sessions, rate limiting	6379 (internal)
backend	FastAPI application server	8000
frontend	Next.js production server	3000
nginx	Reverse proxy + SSL termination	80/443
prometheus	Metrics collection	9090
grafana	Dashboards & visualization	3001
alertmanager	Alert routing	9093
node-exporter	Host metrics	9100
redis-exporter	Redis metrics	9121
postgres-exporter	PostgreSQL metrics	9187
worker	Background task processor	—

v4.0.0 New Features

- Multi-Payment: M-Pesa + Stripe + PayPal + Bank Transfer + Airtel Money + Cryptocurrency (USDT)
- Two-Factor Authentication (TOTP) with QR code setup
- Session Management — view and revoke active sessions
- CSRF Protection with double-submit cookie pattern
- Data Encryption at Rest (Fernet) for PII
- GDPR Compliance — data export and right-to-be-forgotten
- 10 New Pages: About, Privacy, Terms, FAQ, Contact, Blog, Help, Agent Directory, Notifications, Property Compare
- 10 New UI Components: Modal, Select, Tabs, DataTable, Pagination, Skeleton, Tooltip, FileUpload, DatePicker, Alert
- PWA 2.0: Push notifications, background sync, app shortcuts, iOS splash screens
- Refresh Token Auto-Rotation in frontend API client
- Password Strength Meter (zxcvbn)
- Inactivity Auto-Logout
- Next.js Security Headers (CSP, HSTS, X-Frame-Options)

2. All Credentials & Environment Variables

Below is the complete list of all environment variables required to run VESTRA. Copy the backend/.env file and fill in the values marked REQUIRED before deploying.

Database Configuration

Variable	Default	Description	How to Get
TABASE_URL	postgresql+asyncpg://...	PostgreSQL connection string	Your PostgreSQL provider
TABASE_POOL_SIZE	20	Connection pool size	Keep default
TABASE_MAX_OVERFLOW	40	Max overflow connections	Keep default

Redis Configuration

Variable	Default	Description	How to Get
DIS_URL	redis://localhost:6379/0	Redis connection URL	Your Redis provider
DIS_PASSWORD	(empty)	REQUIRED in production	Set strong password
DIS_MAX_CONNECTIONS	50	Max connections in pool	Keep default

Authentication

Variable	Description	How to Generate
CRET_KEY	REQUIRED — 64+ char JWT signing key	python -c "import secrets; print(secrets.token_hex(32
GORITHM	JWT algorithm (HS256)	Keep default
CESS_TOKEN_EXPIRE_MINUTES	Access token lifetime (60)	Keep default
FRESH_TOKEN_EXPIRE_DAYS	Refresh token lifetime (7)	Keep default

M-Pesa Daraja API

To get M-Pesa credentials: 1) Go to https://developer.safaricom.co.ke, 2) Create an account, 3) Create a new app, 4) Select 'Lipa Na M-Pesa Sandbox' or 'Lipa Na M-Pesa Online', 5) Copy Consumer Key and Consumer Secret. 6) Use passkey from Safaricom test credentials page.

Variable	Description	Sandbox Value
ESA_CONSUMER_KEY	Daraja API consumer key	From Safaricom developer portal
ESA_CONSUMER_SECRET	Daraja API consumer secret	From Safaricom developer portal
ESA_SHORTCODE	Business short code	174379 (sandbox)
ESA_PASSKEY	STK Push passkey	bfb279f9aa9bdbcf158e97dd71a467cd2e0c893059b10f78e6b72ada
ESA_CALLBACK_URL	HTTPS callback URL	https://yourdomain.com/api/v1/payments/mpesa/callback
ESA_ENV	sandbox or production	sandbox

Stripe

Variable	Description	How to Get
RIPE_SECRET_KEY	Stripe secret API key	Stripe Dashboard → Developers → API Keys
RIPE_WEBHOOK_SECRET	Webhook signing secret	Stripe Dashboard → Webhooks → Add endpoint

PayPal

Variable	Description	How to Get
YPAL_CLIENT_ID	PayPal REST API client ID	PayPal Developer Dashboard → My Apps & Credentials
YPAL_CLIENT_SECRET	PayPal REST API secret	Same as above
YPAL_ENV	sandbox or live	sandbox

Bank Transfer (Kenyan Banks)

Variable	Description
NK_ACCOUNT_NAME	Account holder name (Vestra Technologies Ltd)
NK_ACCOUNT_NUMBER_KCB	KCB Bank account number
NK_ACCOUNT_NUMBER_EQUITY	Equity Bank account number
NK_ACCOUNT_NUMBER_COOP	Co-operative Bank account number
NK_ACCOUNT_NUMBER_NCBA	NCBA Bank account number
NK_ACCOUNT_NUMBER_ABSA	Absa Bank account number

Airtel Money

Variable	Description
RTEL_CLIENT_ID	Airtel Money API client ID
RTEL_CLIENT_SECRET	Airtel Money API client secret
RTEL_ENV	sandbox or production

Cryptocurrency

Variable	Description	Default
YPTO_WALLET_ADDRESS_USDT	Your USDT wallet address (Polygon)	
YPTO_RPC_URL	Blockchain RPC endpoint	https://polygon-rpc.com
YPTO_ENABLED	Enable/disable crypto payments	false

WhatsApp Business API

Variable	Description
HATSAPP_PHONE_NUMBER_ID	From Meta Business → WhatsApp → API Setup

HATSAPP_ACCESS_TOKEN	Permanent access token from Meta
HATSAPP_VERIFY_TOKEN	Custom token for webhook verification
HATSAPP_BUSINESS_ID	Business account ID from Meta
HATSAPP_APP_SECRET	App secret for payload signing

Email (SMTP)

Variable	Description	Example
TP_HOST	SMTP server hostname	smtp.gmail.com
TP_PORT	SMTP port	587 (TLS) or 465 (SSL)
TP_USER	SMTP username	noreply@vestra.co.ke
TP_PASSWORD	SMTP password or app password	
TP_FROM_EMAIL	Sender email address	noreply@vestra.co.ke
TP_FROM_NAME	Sender display name	Vestra

Security

Variable	Description	How to Generate
CRYPTION_KEY	Fernet key for data encryption	python -c "from cryptography.fernet import Fernet; print(Fe
RNSTILE_SECRET_KEY	Cloudflare Turnstile secret	Cloudflare Dashboard → Turnstile
RNSTILE_SITE_KEY	Cloudflare Turnstile site key	Cloudflare Dashboard → Turnstile
SSION_IDLE_TIMEOUT_MINUTES	Auto-logout after inactivity	30
X_CONCURRENT_SESSIONS	Max sessions per user	5

App & Monitoring

Variable	Description	Default
P_NAME	Application name	Vestra
P_VERSION	Current version	4.0.0
SE_URL	Public URL for email links	https://vestra.co.ke
RS_ORIGINS	Comma-separated allowed origins	https://vestra.co.ke
VIRONMENT	development | staging | production	production
NTRY_DSN	Sentry project DSN	From sentry.io → Settings → Projects
NTRY_TRACES_SAMPLE_RATE	Tracing sample rate	0.1 (10%)

3. Demo Accounts

All demo accounts use the same password: **demo1234**

Role	Email	Name	Capabilities
per Admin	admin@vestra.co.ke	Admin User	Full system access, all admin panels
ent	jane.muthoni@email.com	Jane Muthoni	Property listings, leads, commissions
yer	samuel.njoroge@email.com	Samuel Njoroge	Browse, verify, buy, message
ler	peter.omondi@email.com	Peter Omondi	List properties, manage listings
ndlord	grace.akinyi@email.com	Grace Akinyi	Manage units, tenants, rent collection

4. Deployment Guide

4.1 Local Development

Prerequisites: Python 3.12+, Node.js 20+, PostgreSQL 16, Redis 7

```
# 1. Clone the repository git clone https://github.com/your-org/vestra.git cd vestra # 2. Set up backend cd backend python -m venv venv source venv/bin/activate # Linux/Mac # OR: venv\Scripts\activate # Windows pip install -r requirements.txt # 3. Configure environment cp .env .env.local # Edit SECRET_KEY at minimum # 4. Start PostgreSQL and Redis (Docker recommended) docker run -d --name vestra-postgres -p 5432:5432 \ -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=vestra \ postgres:16-alpine docker run -d --name vestra-redis -p 6379:6379 \ redis:7-alpine --requirepass "" # 5. Run database migrations alembic upgrade head # 6. Seed demo data (optional) python seed_simple.py # 7. Start backend python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload # 8. In another terminal, set up frontend cd ../frontend-build npm install cp .env.example .env.local # Configure NEXT_PUBLIC_API_URL # 9. Start frontend npm run dev # 10. Open browser open http://localhost:3000
```

4.2 Docker Production Deployment

```
# 1. Fill in ALL production environment variables # Edit vestra/.env with all production values # Generate encryption key: python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())" # 2. Build Docker images cd vestra docker-compose build # 3. Start all services (detached) docker-compose up -d # 4. Run database migrations docker-compose exec backend alembic upgrade head # 5. Verify deployment curl http://localhost/health curl http://localhost/metrics # 6. Check logs docker-compose logs -f backend docker-compose logs -f nginx # 7. Stop all services docker-compose down
```

4.3 Cloud Deployment Options

Platform	Pros	Best For	Config Files
.io	Auto-scaling, global edge	Production apps	fly.toml (included)
lway	Simple deploy, managed DB	Quick setups	railway.json (included)
nder	Free tier, easy SSL	Small/medium apps	render.yaml (included)
/S ECS	Enterprise scale, full control	Large deployments	Custom Docker
italOcean	Simple droplets, managed DB	African latency	Docker Compose

5. System Architecture

Backend Architecture

The backend is a FastAPI application with 25+ route modules, 14 model files (40+ database tables), 30+ service modules, and a comprehensive middleware stack. Key architectural patterns:

- Dual Router: All endpoints at /api/v1/* (canonical) AND /api/* (legacy with deprecation headers)
- Fire-and-Forget: Background tasks via asyncio.create_task() for analytics, notifications, email
- Redis-First: Database-heavy endpoints cached in Redis with short TTLs
- Subscription-Gating: Role-based access with per-tier limits enforced at the service layer
- M-Pesa Primary: Deep integration with Safaricom M-Pesa for STK Push payments
- AI-Powered: Custom rule-based AI engine for property valuation, trust scoring, NL search

Security Architecture

- JWT with IP binding and refresh token rotation
- TOTP-based Two-Factor Authentication (pyotp)
- CSRF Protection via double-submit cookie pattern
- Rate Limiting: Redis-backed sliding window (10/min auth, 120/min general)
- Account Lockout: 5 failed attempts → 15-minute lockout via Redis
- M-Pesa Callback: Safaricom IP whitelist + HMAC verification + Redis replay protection
- Stripe Webhook: Signature verification + idempotency keys
- Data Encryption: Fernet encryption for PII (national_id, phone) at rest
- Security Headers: CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy

Frontend Architecture

The frontend is a Next.js 16 App Router application with 60+ pages, 30+ components, and comprehensive PWA support. Key patterns:

- Role-Based Dashboards: 6 distinct dashboards (buyer, seller, landlord, tenant, agent, admin)
- Auth Flow: Zustand persist + AuthGuard (6-layer verification) + AuthInit
- Design System: Custom emerald/amber color palette, glass morphism, 16 custom animations
- API Client: Axios with retry logic, correlation IDs, token management, auto-refresh
- PWA: Service worker with caching, push notifications, app shortcuts, offline support
- Theme: System-aware dark/light mode with smooth transitions

6. API Endpoints Reference

All endpoints are available at /api/v1/* (canonical) and /api/* (legacy, deprecated). Legacy endpoints include X-API-Deprecated and Sunset headers.

Auth (15+ endpoints)

Endpoint	Description
ST /auth/register	Register with email, password, CAPTCHA, referral
ST /auth/login	Login — returns JWT tokens
ST /auth/login/form	OAuth2 form for Swagger UI
T /auth/me	Get current user profile
T /auth/me	Update current user profile
ST /auth/change-password	Change password (authenticated)
ST /auth/forgot-password	Send password reset email
ST /auth/reset-password	Reset password with token
ST /auth/verify-email	Verify email address
ST /auth/resend-verification	Resend verification email
ST /auth/refresh	Refresh access token (rotation)
ST /auth/logout	Revoke all refresh tokens
T /auth/referral-code	Get user's referral code and stats
ST /auth/send-otp	Send OTP via WhatsApp
ST /auth/verify-otp	Verify OTP, create/get user
ST /auth/upgrade-role	Upgrade buyer to seller/agent/landlord
ST /auth/2fa/setup	Generate TOTP secret + QR code (NEW v4)
ST /auth/2fa/verify	Verify & enable 2FA (NEW v4)
ST /auth/2fa/disable	Disable 2FA (NEW v4)
T /auth/sessions	List active sessions (NEW v4)
LETE /auth/sessions/{id}	Revoke session (NEW v4)

Properties (12+ endpoints)

Endpoint	Description
ST /properties	Create property listing
T /properties	Search/filter properties
T /properties/ai-search	Natural language AI search
T /properties/my	Current user's properties

T /properties/{id}	Property detail with view tracking
T /properties/{id}/seo	SEO metadata (OG, Twitter, JSON-LD)
T /properties/{id}	Update property
LETE /properties/{id}	Soft delete property
ST /properties/{id}/publish	Publish property
ST /properties/{id}/feature	Purchase featured placement
T /properties/listing-fee/info	Usage and limits

Payments (Multi-Provider — v4 Expanded)

Endpoint	Description
ST /payments/mpesa/initiate	Initiate M-Pesa STK Push
ST /payments/mpesa/callback	M-Pesa callback (IP whitelist + HMAC)
T /payments/status/{id}	Payment status
T /payments/my	User's payment history
ST /payments/stripe/callback	Stripe webhook
ST /payments/{id}/refund	Process refund
ST /payments/initiate/paypal	Initiate PayPal payment (NEW v4)
ST /payments/paypal/callback	PayPal webhook (NEW v4)
ST /payments/initiate/bank	Generate bank transfer reference (NEW v4)
ST /payments/bank/confirm	Admin confirm bank deposit (NEW v4)
ST /payments/initiate/crypto	Generate crypto payment address (NEW v4)
ST /payments/crypto/callback	Crypto confirmation webhook (NEW v4)
ST /payments/initiate/airtel	Initiate Airtel Money (NEW v4)
T /payments/methods	List available payment methods (NEW v4)

Verification (6 endpoints)

Endpoint	Description
ST /verify/request	Initiate verification with M-Pesa payment
ST /verify/run/{id}	Run AI verification
T /verify/status/{id}	Get verification status
T /verify/property/{id}	All verifications for property
ST /verify/documents/upload	Upload property documents
T /verify/admin/review/{id}	Admin review verification

Rentals (20+ endpoints)

Endpoint	Description
T /rentals/dashboard	Landlord portfolio dashboard
ST /rentals/units	Create rental unit
T /rentals/units	List landlord's units
ST /rentals/tenants	Add tenant with lease
T /rentals/tenants	List landlord's tenants
ST /rentals/rent/generate-bills	Generate monthly rent bills
ST /rentals/rent/request-payment/{id}	M-Pesa STK Push for rent
ST /rentals/rent/record-payment/{id}	Manual rent payment record
ST /rentals/maintenance	Submit maintenance request
T /rentals/maintenance/{id}	List maintenance for unit
T /rentals/maintenance/{id}	Update maintenance status
ST /rentals/arrangements/request	Request payment arrangement
T /rentals/arrangements/{id}/approve	Approve arrangement
T /rentals/collection-config/{lease_id}	Get rent collection config

Admin (20+ endpoints)

Endpoint	Description
T /admin/stats	Dashboard stats (15+ queries, 2min cache)
T /admin/users	User management
T /admin/payments	Payment management
T /admin/revenue/summary	Revenue analytics
T /admin/audit-logs	Audit trail
T /admin/fraud-reports	Fraud report list
T /admin/kyc/pending	Pending KYC reviews
T /admin/verifications/queue	Verification review queue
ST /admin/verifications/bulk-review	Bulk verification review

7. Database Schema

PostgreSQL 16 with 40+ tables across 14 model files.

Table	Purpose	Key Columns
ers	Core user accounts	id, email, phone, hashed_password, role, is_active, is_verified, is_kyc_verified, tw
perties	Property listings	id, owner_id, title, description, type, listing_type, status, price, currency, bedrooms
ent_profiles	Agent details	user_id, agency_name, license_number, years_experience, badge_level, rating,
ifications	Property verification	id, property_id, status, fraud_risk_score, trust_score, ai_recommendation
cuments	Property documents	id, property_id, document_type, file_path, is_verified
yments	Payment transactions	id, user_id, amount, method, purpose, status, mpesa_*, stripe_*, provider_*
bscriptions	User subscriptions	id, user_id, tier, status, amount, period_start, period_end, auto_renew
ifications	In-app notifications	id, user_id, type, title, body, action_url, is_read, channel
ssages	User-to-user messages	id, sender_id, receiver_id, property_id, subject, body, is_read
_verifications	KYC identity checks	id, user_id, status, id_type, id_number, id_front_url, selfie_url
tal_units	Landlord rental units	id, landlord_id, property_id, monthly_rent_kes, deposit_kes, is_occupied
ants	Rental tenants	id, unit_id, full_name, phone, move_in_date, rent_due_day
ses	Tenant leases	id, unit_id, tenant_id, status, start_date, end_date, monthly_rent_kes
t_payments	Rent payment records	id, tenant_id, amount_kes, status, due_date, paid_date, mpesa_receipt
intenance_requests	Maintenance tracking	id, unit_id, tenant_id, priority, status, category
crow_transactions	Property purchase escrow	id, property_id, buyer_id, seller_id, amount_kes, deposit_reference, payment_ref
putes	User disputes	id, reporter_id, property_id, category, status, resolution
ud_reports	Fraud reporting	id, reporter_id, reported_phone, reported_email, status
iews	User reviews	id, reviewer_id, subject_id, property_id, rating, is_verified_transaction
errals	Referral tracking	id, referrer_id, referred_user_id, referral_code, status, rewards_earned
e_chain_blocks	Title history blockchain	id, property_id, block_index, event_type, previous_hash, block_hash
_keys	Enterprise API keys	id, user_id, key_hash, scopes, rate_limit_per_min
phooks	Enterprise webhooks	id, user_id, url, secret, events, is_active
upons	Discount coupons	id, code, discount_type, discount_value, max_uses
er_events	Analytics events	id, user_id, session_id, event_type, event_data
arch_analytics	Search tracking	id, user_id, query, filters_applied, results_count

8. Frontend Routes

Next.js 16 App Router with 60+ pages.

Public Pages

- / (Homepage — AI search, features, testimonials)
- /market (Property search — grid/list/city views)
- /verify (Property verification flow)
- /agents (Agent directory)
- /agents/directory (Browse verified agents — NEW v4)
- /properties/[id] (Property detail page)
- /properties/compare (Side-by-side comparison — NEW v4)

Auth Pages

- /auth/login
- /auth/register
- /auth/forgot-password

Dashboard — Buyer

- /dashboard/buyer (Saved searches, recently viewed)
- /dashboard/buyer/favorites
- /dashboard/buyer/escrow

Dashboard — Seller

- /dashboard/seller (Listings, views, inquiries)
- /dashboard/seller/analytics

Dashboard — Landlord

- /dashboard/landlord (Units, occupancy, collections)
- /dashboard/landlord/tenants
- /dashboard/landlord/maintenance

Dashboard — Tenant

- /dashboard/tenant (Current rental, pay rent)
- /dashboard/tenant/rent
- /dashboard/tenant/receipts
- /dashboard/tenant/maintenance
- /dashboard/tenant/discover

Dashboard — Agent

- /dashboard/agent (Leads, listings, deals)

- /dashboard/agent/leads
- /dashboard/agent/commissions

Admin Pages

- /admin (Dashboard overview)
- /admin/users
- /admin/properties
- /admin/payments
- /admin/verifications
- /admin/kyc
- /admin/fraud
- /admin/disputes
- /admin/monitoring

User Pages

- /account
- /settings
- /settings/security
- /settings/kyc
- /messages
- /notifications (NEW v4)
- /wallet
- /subscription
- /subscription/manage

Content Pages (NEW v4)

- /about
- /privacy
- /terms
- /faq
- /contact
- /blog
- /blog/[slug]
- /help

Enterprise

- /enterprise
- /enterprise/keys

9. Payment Integration Guide

9.1 M-Pesa Daraja API

M-Pesa is the primary payment method for Kenyan users. The system uses Safaricom's Daraja API with STK Push (Lipa Na M-Pesa Online) for payments.

- Register at https://developer.safaricom.co.ke
- Create a new app → Select 'Lipa Na M-Pesa Sandbox'
- Copy Consumer Key and Consumer Secret
- Set MPESA_ENV=sandbox for testing, =production for live
- Sandbox test phone: 254708374149
- Sandbox passkey: bfb279f9aa9bdbcf158e97dd71a467cd2e0c893059b10f78e6b72ada1ed2c919
- Production: Submit for Go-Live review with business docs
- IMPORTANT: Set the callback URL to HTTPS. Safaricom calls this when payment completes.
- M-Pesa IPs must be whitelisted (they're checked in the callback handler).

9.2 Stripe

- Register at https://dashboard.stripe.com
- Get API keys from Developers → API Keys
- Set STRIPE_SECRET_KEY=sk_live_... for production or sk_test_... for testing
- Create webhook endpoint for /api/v1/payments/stripe/callback
- Get webhook signing secret and set STRIPE_WEBHOOK_SECRET
- Test cards: 4242 4242 4242 4242 (Visa), 5555 5555 5555 4444 (Mastercard)

9.3 PayPal

- Register at https://developer.paypal.com
- Create REST API app → Get Client ID and Secret
- Set PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, PAYPAL_ENV=sandbox
- Configure webhook URL for payment capture events
- Switch to PAYPAL_ENV=live for production

9.4 Bank Transfer

- Open business accounts with Kenyan banks (KCB, Equity, Co-op, NCBA, Absa)
- Set BANK_ACCOUNT_NAME and account numbers in .env
- When user selects bank transfer, system generates a unique reference number
- User transfers to the bank account with the reference as description
- Admin confirms payment manually via /admin/payments after verifying bank deposit
- No API integration needed — fully manual confirmation flow

9.5 Airtel Money

- Contact Airtel Kenya for merchant API access
- Get Client ID and Client Secret for API authentication
- Set AIRTEL_CLIENT_ID, AIRTEL_CLIENT_SECRET, AIRTEL_ENV=sandbox

- Similar flow to M-Pesa — push notification to user's phone

9.6 Cryptocurrency (USDT on Polygon)

- Create a USDT wallet on Polygon network (use MetaMask, Rabby, or similar)
- Set CRYPTO_WALLET_ADDRESS_USDT to your Polygon wallet address
- Set CRYPTO_ENABLED=true to enable crypto payments
- System generates a unique payment reference for each transaction
- Payments confirmed via Polygon RPC block scanning
- Polygon chosen for low fees (~$0.01 per transaction vs $5-50 on Ethereum)
- USDT stablecoin chosen to avoid price volatility

10. Maintenance & Operations

Database Backups

```
# Automated daily backup (add to crontab) 0 2 * * * pg_dump -h localhost -U postgres vestra |
gzip > /backups/vestra_$(date +\%Y\%m\%d).sql.gz # Restore from backup gunzip -c
vestra_20260621.sql.gz | psql -h localhost -U postgres vestra # Point-in-time recovery
(requires WAL archiving) # See PostgreSQL docs:
https://www.postgresql.org/docs/16/continuous-archiving.html
```

Redis Persistence

Redis is configured with AOF (Append Only File) persistence and LRU eviction (maxmemory 256MB). AOF files are stored at /data/appendonly.aof. For production, consider Redis Sentinel or Cluster for high availability.

Log Management

All services log structured JSON to stdout/stderr with rotation configured in docker-compose.yml. Logs include correlation IDs for tracing requests across services. For production, ship logs to a centralized system (ELK, Loki, or CloudWatch).

Monitoring Checklist

- Grafana: http://your-server:3001 (admin / configured password)
- Prometheus: http://your-server:9090
- Alertmanager: http://your-server:9093
- Key metrics to watch: API latency p95, error rate, DB connections, Redis hit rate, M-Pesa callback failures
- Set up alerts for: 5xx errors > 1%, DB connections > 80%, Redis memory > 90%, disk > 85%

Scaling

- Backend: Increase gunicorn workers (2-4 x CPU cores) or add more containers
- Database: Add read replicas for search-heavy workloads. Consider connection pooling (PgBouncer)
- Redis: Upgrade to larger instance or use Redis Cluster for >256MB data
- CDN: Put Cloudflare in front for static assets, image caching, DDoS protection
- Queue: Use dedicated task queue (Celery/Redis) for heavy background jobs

Security Maintenance

- Run monthly dependency scan: pip-audit, npm audit
- Keep Python, Node.js, PostgreSQL, Redis updated to latest patch versions
- Rotate SECRET_KEY and ENCRYPTION_KEY every 90 days
- Review audit logs weekly for suspicious activity
- Test backup restoration quarterly
- Run OWASP ZAP scan before major releases

11. Troubleshooting Guide

Database connection refused

- Check if PostgreSQL is running: docker-compose ps postgres
- Verify DATABASE_URL in .env has correct host/port/credentials
- If using Docker, services must be on the vestra_internal network
- Check if port 5432 is already in use: lsof -i :5432

Redis connection failed

- Check if Redis is running: docker-compose ps redis
- Verify REDIS_URL and REDIS_PASSWORD in .env
- Test connection: redis-cli -h localhost -p 6379 -a PING
- Redis caching degrades gracefully — app works without Redis but without caching

M-Pesa callback not receiving payments

- Verify CALLBACK_URL is HTTPS (Safaricom requires HTTPS in production)
- Check that callback URL is publicly accessible (not localhost)
- Verify Safaricom IPs are in the whitelist (payments.py)
- Check Docker logs: docker-compose logs backend | grep mpesa
- Test with Safaricom's callback simulator in developer portal

Emails not sending

- Verify SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD are correct
- For Gmail: Use App Password (not account password), enable 2FA first
- Check spam folder at destination
- Test: docker-compose exec backend python -c "from app.services.email_service import send_email; ..."

Frontend build fails

- Check Node.js version: node --version (needs 20+)
- Delete node_modules and package-lock.json, then npm install
- Run npm run typecheck to find TypeScript errors
- Check if scripts/generate-pwa-icons.js exists and is executable

Docker containers keep restarting

- Check logs: docker-compose logs
- Verify .env has all required variables (especially SECRET_KEY)
- Check disk space: df -h
- Check memory: docker stats

API returning 500 errors

- Check backend logs: docker-compose logs backend --tail=100
- Check Sentry dashboard for error details (if configured)
- Check database migrations are applied: docker-compose exec backend alembic current
- Verify all environment variables are set correctly

CORS errors in browser

- Verify CORS_ORIGINS includes your frontend URL
- For development: CORS_ORIGINS=http://localhost:3000
- For production: CORS_ORIGINS=https://vestra.co.ke,https://www.vestra.co.ke

12. Support & Contact

For technical support, deployment assistance, or feature requests:

Contact Method	Details
pport Email	support@vestra.co.ke
ergency Phone	+254 700 000 000
ice Location	Nairobi, Kenya
stem Version	v4.0.0
cument Generated	June 21, 2026 at 14:21
nerated By	Claude Code — Anthropic
pository	github.com/your-org/vestra

Built for Kenya. Trusted by Africa. Powered by AI.